From RL Papers to Framework Code: Inclusion and Review in Seven Open-Source Training Frameworks

Taiheng Pan
The University of Melbourne
taiheng.pan@student.unimelb.edu.au

Abstract

Practitioners use reinforcement learning methods for language models through training frameworks, so a method that a framework includes is one configuration change away for its users. We measure how often this happens. We trace seven open-source frameworks for RL post-training through their commit histories and link them to 6,939 screened arXiv papers from January 2025 to June 2026. Of 1,598 papers that propose a general RL training method, 37 reached at least one framework. Across five quarterly cohorts of papers, the number included within 180 days stayed between 5 and 7 while the cohorts grew from 72 to 386 papers. Compared with papers from the same quarters, included papers more often trained policies of 30B parameters or more, at 41 against 10 percent, and more often changed a coefficient, clip, mask or weight in the existing loss, at 29 against 9 percent. Both gaps hold without papers from the companies that maintain the frameworks, while code release was equally common in both groups. The frameworks' issues and pull requests show that few methods are turned away in public. More than nine in ten of the general methods no framework included have no thread there that cites or names them, and many proposals that do appear stay open without a reply. Two further records, the frameworks' issues and the technical reports of 13 large open-weight models, show that users rarely report training failures in public and that the reports state 8 of the 13 widely included methods. We release the data as a tracker that links training failures to framework implementations and lists each framework's contribution rules and earlier proposals.

1 Introduction

GSPO [88] reached TRL 3 days after its first arXiv version, and verl, OpenRLHF, ROLL and slime followed within a week. All seven frameworks in this study include it today. In the same month, 43 other papers proposed a general RL training method for language models, and only 2 of them reached any of the seven frameworks.

Research on RL post-training has grown fast: in each month of the first half of 2026, at least 144 papers in our pool proposed an RL training method. Practitioners meet these methods through the frameworks they train with, and a framework decides whether a method is one configuration change away or a separate project. Publishing a method, releasing its code and having a framework include it are three separate outcomes, and the last one has not been measured.

We measure framework inclusion. A method counts as included when its implementation reaches the main branch of a tracked framework, and we record the tier of each inclusion: a default, a selectable algorithm, a switch, experimental code or an example recipe. Inclusion describes the code that frameworks ship, and Section 11 sets out its limits.

Technical report, version 1, 20 October 2026. Data and code: https://github.com/pth2002/rl-method-tracker

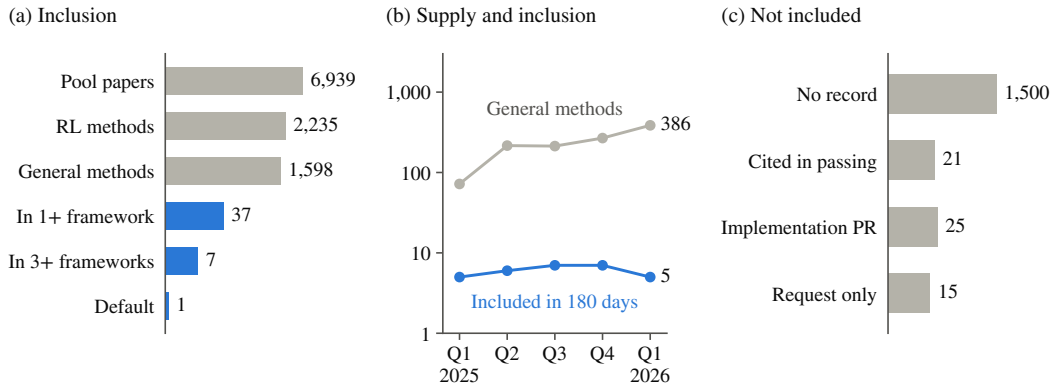


Figure 1: Inclusion, supply against intake, and the public record of the rest. Unit: paper. (a) Papers from January 2025 to June 2026: the screened pool, papers that propose an RL training method, general methods, and general methods by the strongest inclusion they reached, on a log scale. (b) General methods by the quarter of their first arXiv version in grey, and those a framework included within 180 days of that version in blue, for the quarters with 180 days of exposure before the snapshot for every paper, on a log scale. (c) General methods that no framework included, grouped by the threads in the seven frameworks that link them, give their identifier or quote their title, on a log scale. These groups are categories and not stages.

Inclusion is rare: only 37 of 1,598 papers that propose a general method reached one of seven frameworks (2.3 percent). The included methods differ from the rest in two ways (Section 6). Their papers more often trained policies of 30B parameters or more, at 41 against 10 percent, and more often changed a coefficient, clip, mask or weight in the existing loss, at 29 against 9 percent. Code release was equally common, at 44 percent of included papers and 44 percent of others. The two gaps persist without model reports, within papers that have an industry author, and without papers from the companies that maintain the frameworks. They remain associations, and Section 6 sets out their limits.

The intake did not keep pace with the supply. Across five quarterly cohorts, the number of general methods a framework included within 180 days stayed between 5 and 7, while the cohorts grew from 72 to 386 papers, a 5.4-fold rise (Figure 1b). Counted by the date of the landing instead, first landings of general methods rose from 3 a quarter at the start of 2025 to 10 at its end. The frameworks took in more methods as the supply grew, but a shrinking share of each cohort.

Few of the other methods were turned away in public. We collected the issues, pull requests and discussions of all seven frameworks and searched them for every general method in the pool. Maintainers declined or redirected proposals for only 12 of the 1,561 general methods that no framework included. Most never came up in a framework's issues or pull requests at all: 96 percent have no thread that cites them (Figure 1c), and a search by method name lowers this only to 95 percent. Many proposals that do appear stay open without a reply. Merged pull requests also drew concerns, mostly about design and tests, and their authors often revised the code in response. Concerns about demand and maintenance burden appear mostly in proposals that were not merged, and most of those come from one framework (Section 7).

Two further records place inclusion next to users and to large training runs. In the frameworks' issues and discussions, 48 of the 7,075 threads opened since January 2025 report a training failure, and the failures they name are distributed differently from those that paper abstracts address (Section 8). Technical reports of 13 open-weight models with 100B parameters or more state 8 of the 13 widely included methods, describe variants of others under their own names, and are the source of 2 (Section 9).

We release the data and code as a tracker. Its index links training failures, such as entropy collapse or a mismatch between rollout and training probabilities, to the implementations and settings in each framework. A second page gives each framework's contribution rules, the concerns its reviewers raised and the outcome of earlier proposals, as a starting point for authors who want a framework to include their method.

2 Related work

Studies of industry and academia in AI mostly work at the level of papers and topics. They track the growing share of industry in AI research [2] and the association between code availability and citation counts [90]. Forks of the repositories that accompany ML papers rarely change the code, and fewer send changes back [5]. These measures capture attention among researchers but do not observe whether a method reaches the code that trains models.

A second line asks whether reported gains hold up. Re-evaluations in recommendation [12] and metric learning [50] found that many improvements vanished under careful comparison, and broader critiques point to benchmark selection [15] and scholarship practice [38]. Implementation details matter as much in policy-gradient methods: code-level choices explain much of the reported gap between PPO and TRPO [17], and an implementation guide lists 37 such details for PPO alone [29]. For RL with language models, controlled studies compare tricks under one implementation [44] and test methods at large compute [31]. These studies ask whether a method works. We ask whether frameworks include it, and we read the answer from their code.

A third line studies the decisions of open-source projects on outside contributions. Only a few factors drive the decision to merge a pull request, and few rejections are technical [23]. Project managers weigh technical signals together with their social ties to the submitter [72], and integrators report trouble keeping quality high and ranking the contributions they receive [24]. Reviewers also look at signals beyond the code [19], and ties across an ecosystem shift acceptance [46]. These studies explain decisions about code contributions in general. We follow a narrower object: a method from a paper, traced from its evidence to its implementation in a training framework and to the public record of each proposal.

3 Measuring inclusion

Frameworks. We study verl [64] with its recipe repository, TRL [74], OpenRLHF [27], NeMo-RL [51], ROLL [76], slime [92] and AReaL [20]. All seven meet four criteria: they are built for RL post-training of language models, run their own training loop with several selectable RL algorithms, are developed in public, and have at least 2,000 GitHub stars. Four other frameworks meet the same criteria: open-instruct, SkyRL, prime-rl and RLinf (Table 10). At their latest commits they implement 2 general methods that none of the seven include, so adding them would raise the inclusion rate from 2.3 to 2.4 percent. The snapshots of six frameworks date from 13 to 17 October 2026. OpenRLHF had no main-branch commit after 17 September 2026, so its snapshot is older (Table 10).

Algorithm locations. Each framework exposes its RL algorithms in a few places. verl registers advantage estimators and policy losses by name, while TRL selects GRPO losses through a loss-type field and keeps newer trainers in an experimental folder. AReaL ships one example configuration per algorithm, and the other frameworks use command-line choices and configuration switches. We list these places for each framework, parse them at every commit on the main branch, and check the result against each framework's algorithm documentation at the snapshot.

Tiers. We record the tier of each inclusion. A *default* is on in a run that keeps every setting. A *conditional default* is on by default once a related feature is enabled, such as generation with vLLM. A *core* option is a selectable algorithm in the main code path, and a *switch* modifies the main algorithm. *Experimental* code and example *recipes* sit outside the main path.

From options to methods. Frameworks name the same method differently: clip-higher [84] appears as `clip_ratio_high` in verl, `epsilon_high` in TRL, `eps_clip_low_high` in OpenRLHF and `eps-clip-high` in slime. We map every option name to one entry in a catalog of 65 methods and check each source against its arXiv record. The parser reports every option it detects without a mapping, and the build fails until the option is mapped. An event is the addition or removal of an option, or a change of its default, on a main branch, and the history of the seven frameworks yields 426 events. Implementations outside the parsed places are the target of the documentation check and of the citation audit below.

Citation audit. Parsing misses an implementation that sits under a name the catalog does not map, in a place we do not parse, or in a change to existing code. We therefore searched the full main-branch history of each framework for the arXiv identifiers of pool papers and kept every file that cites one. Framework files cite 51 general methods, including all 28 that parsing had found. For 9 of the other 23, a citation sits next to an implementation, and we added them to the catalog. Two of them add no option: a fix to the KL gradient in NeMo-RL and the trajectory-level agent pipeline of ROLL. We record such implementations from their commits, so the implementation stays the test of inclusion. The remaining papers appear in lists of projects built on a framework, in reference lists of technical reports, or as background (Appendix E).

Dates. A date is the commit date on the main branch, an approximation of the day a change became available there. An option first seen in a repository's root commit, in a bulk code release, or in a commit that renamed an existing option has an unknown start, so we treat such a date as an upper bound. A *landing* is the first commit that brings a method into one framework, and the *first landing* of a method is its earliest landing in any framework. A later exact date from another framework never replaces an earlier upper bound. Lags run from the first arXiv version of the source, or from the post date for blog sources.

People. For each landing we compare the git author of the introducing commit with the author list of the source. The comparison has four outcomes: a paper author, a named person outside the list, a handle we cannot resolve, and a source without an author list. When the commit names a pull request, we also record who opened it, who merged it and who reviewed it. We call a person a prior merger of a framework once they have merged a pull request there, comparing UTC timestamps with the event in question. Prior mergers stand in for maintainers, but the proxy misses reviewers without merge rights and people before their first merge, and it keeps people who have since left. We also match the affiliations on each paper's first page against the company that maintains each framework, such as ByteDance for verl and NVIDIA for NeMo-RL. OpenRLHF has no maintaining company.

4 Measuring the supply of methods

Screening. Keyword search through the arXiv API misses papers that typeset method names in LaTeX. The DAPO abstract, for example, writes its name through bold initials. We therefore harvest the complete computer science metadata through the arXiv OAI-PMH interface and keep the 139,135 records in cs.LG, cs.CL or cs.AI with identifiers from 2501 to 2606. A paper enters the pool when its title or abstract mentions language models and reinforcement learning, including the abbreviations RL and RFT. The pool holds 6,939 papers and contains 45 of the 46 catalog sources from this window. This check covers included sources only, so the pool is a screened set and not a complete census of RL papers.

Labels. Each paper receives one of seven labels from its title and abstract under fixed rules (Appendix A). The label of interest marks a new or modified training-time RL component. We call such a method *general* when the component is the paper's main contribution and is presented for RL with language models at large. We call it *embedded* when it serves a paper about one task, domain or application. The denominator is the set of general methods.

Checks. Three checks address recognition, stability and the general rule. For recognition, the included sources were mixed into the first batches without marking, and 29 of the 30 sources not used as examples in the labelling rules were labelled as methods. The miss is a technical report that describes its method only in the body. For stability, 140 papers were labelled twice in separate batches and kept the same primary label 129 times. For the general rule, a stratified sample of 200 papers was reviewed manually. Review and labels agree on 188 of them for the split between general methods and all other papers, and weighting by the three label strata puts the number of general methods at about 1,590 (Appendix B).

Model reports. Technical reports for new models often describe their RL changes only in the body, so we read 70 randomly drawn reports in full. The sample holds 30 reports labelled as methods and 40 labelled otherwise, and it leaves out reports with included methods. A transferable RL component appears in the body of 25 reports of the first group and 6 of the second. The general rule asks for more

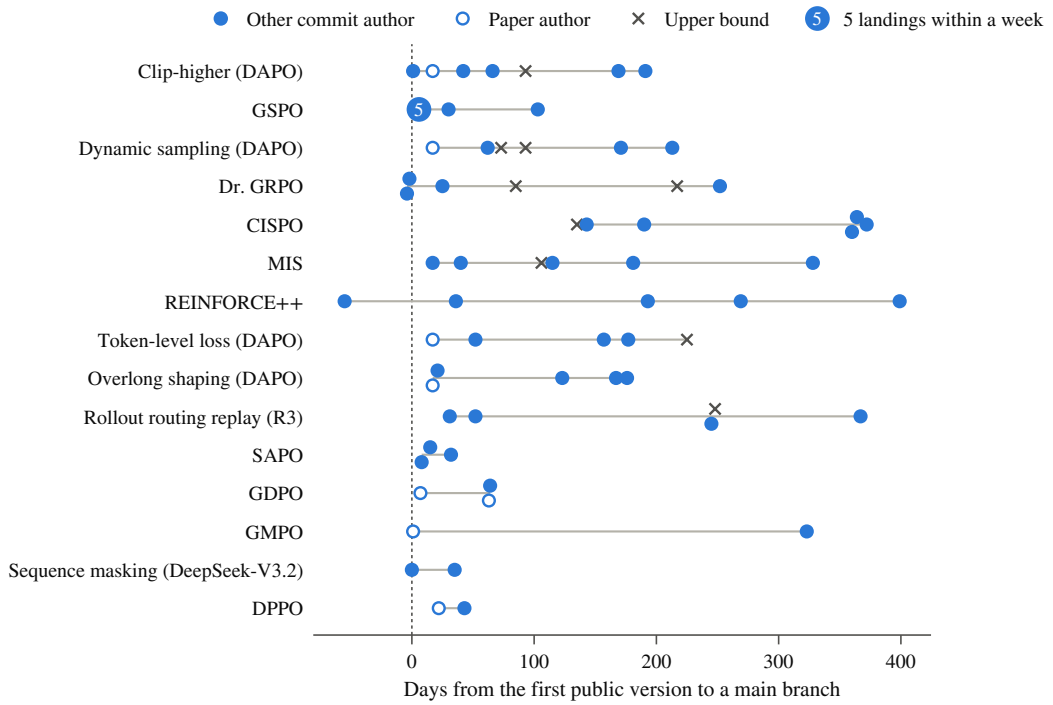


Figure 2: Time from first public version to a main branch. Unit: catalog method with a dated source from 2025 or later that reached two or more frameworks. Figure 5 in the appendix shows every method. Each mark is one landing. Hollow dots mark commits by a paper author. Crosses mark landings in a bulk release or a renaming commit, and their dates are upper bounds. Landings within a few days of each other are spread vertically within their row. The larger mark with a number counts landings too close to draw apart: five frameworks took in GSPO between 3 and 7 days after its first version.

than a transferable component, so these counts bound the effect of abstract-only labels from above. Counting every report with such a component as a general method would raise the denominator by at most about 13 percent and lower the inclusion rate from 2.3 to 2.1 percent.

5 Inclusion rate

Few general methods reach a framework. Figure 1a follows the pool from all RL papers to framework defaults: 37 of 1,598 general methods reached at least one framework, a rate of 2.3 percent. Three or more frameworks include 7 of them today, and 1 is an unconditional default.

Many methods were brought in by people outside the author list. For 18 included papers, at least one landing came from a named commit author outside the paper's author list. Another 12 reached frameworks only through commits by their authors, and the remaining 7 only through unresolved handles or sources without an author list.

Another 8 included papers fall outside the general rule from their abstracts. Most are model reports or system papers with an included component in the body (Appendix D). Adding them gives 45 included papers out of 1,606, with 9 in three or more frameworks.

Exposure time matters: papers from 2025 had at least nine months before the snapshot, and 31 of 769 reached a framework. Papers from the first half of 2026 had less time, and 6 of 829 did. With a common window, 30 of the 1,155 general methods with at least 180 days of exposure were included within 180 days. Counted by the quarter of the landing instead, first landings of general methods rose from 3 in the first quarter of 2025 to 10 in the fourth, and 9 landed in the first quarter of 2026. The intake grew while the share of each cohort included within 180 days fell from 6.9 to 1.3 percent.

Inclusion is fast when it happens. Among the 54 catalog methods with a source from 2025 or later, 49 have an exact first landing, and their median time from the first arXiv version to a main branch is

Table 1: Recent methods included in three or more frameworks. Unit: catalog method. Days run from the first public version of the source to the first landing on a main branch. A $\leq$ sign marks an upper bound, and negative values mean the commit predates the arXiv version. Of these, truncated importance sampling (TIS) is a conditional default in TRL and token-level loss aggregation is a default in TRL.

Method	Source	First public	Frameworks	Days	Failure it addresses
Clip-higher	[84]	2025-03-18	7	1	entropy collapse
GSPO	[88]	2025-07-24	7	3	MoE instability, extreme ratios
Dynamic sampling	[84]	2025-03-18	6	17	groups without signal
Dr. GRPO	[43]	2025-03-26	6	-4	length bias
CISPO	[49]	2025-06-16	6	$\leq$135	entropy collapse
Truncated importance sampling (TIS)	[82]	2025-08	6	–	rollout-training mismatch
Masked importance sampling (MIS)	[37]	2025-10-21	6	17	rollout-training mismatch
REINFORCE++	[28]	2025-01-04	5	-55	value-model cost, gradient variance
Token-level loss aggregation	[84]	2025-03-18	5	17	length bias
Overlong filtering and penalty	[84]	2025-03-18	5	17	truncation noise
Rollout routing replay	[45]	2025-10-13	5	31	MoE instability, rollout-training mismatch
SAPO	[21]	2025-11-25	3	8	extreme ratios
GDPO	[41]	2026-01-08	3	7	multi-reward collapse

30 days (Figure 2). Another 4 have only an upper bound, and 1 blog source is dated only by month. Counted once per source paper, the median is 30.5 days over 42 papers, and in 5 cases the commit predates the arXiv version. These medians describe included methods only. Among methods in three or more frameworks, CISPO [49] is the slowest: it reached its first framework at most 135 days after its source, and TRL added it later in a commit named after a large comparison study [31].

Authorship changes with breadth. Among the 38 recent methods found in exactly one framework, 13 first landed through a commit by a paper author, most of them as example recipes. Methods in three or more frameworks have 70 landings, and 40 of these came from named people outside the author list. Paper authors made 6, unresolved handles 18, and the remaining 6 belong to blog sources without an author list.

Defaults are rare. TRL turned on token-level loss aggregation in August 2025 and sets the KL coefficient to zero by default. It also enables truncated importance sampling by default once generation with vLLM is switched on, and applies the importance-weighted KL correction of DeepSeek-V3.2 [14] once the KL coefficient is non-zero. NeMo-RL applies the corrected KL gradient of Tang and Munos [69] whenever its KL penalty is non-zero (Section 7). Table 1 lists the recent methods included in three or more frameworks.

6 Features of included methods

Figure 3 compares the 31 included general methods from 2025 with 150 general methods from the same year that no framework included. The controls were drawn at random within each quarter, and fields were coded from the first arXiv version of each paper without access to inclusion status (Appendix C). The citation audit found 8 of the included papers later, and a second reader recoded these papers blind. The comparison uses the blind codes, and Table 7 gives the gaps without them. The comparison is descriptive, because scale, resources, team composition and breadth of experiments can vary together.

The figure shows every coded field. We single out two of them because they have the largest ratio between the groups among the fields that at least five papers show in each group. Mixture-of-experts policies differ by a larger ratio, but they appear in only 3 included papers and 1 other.

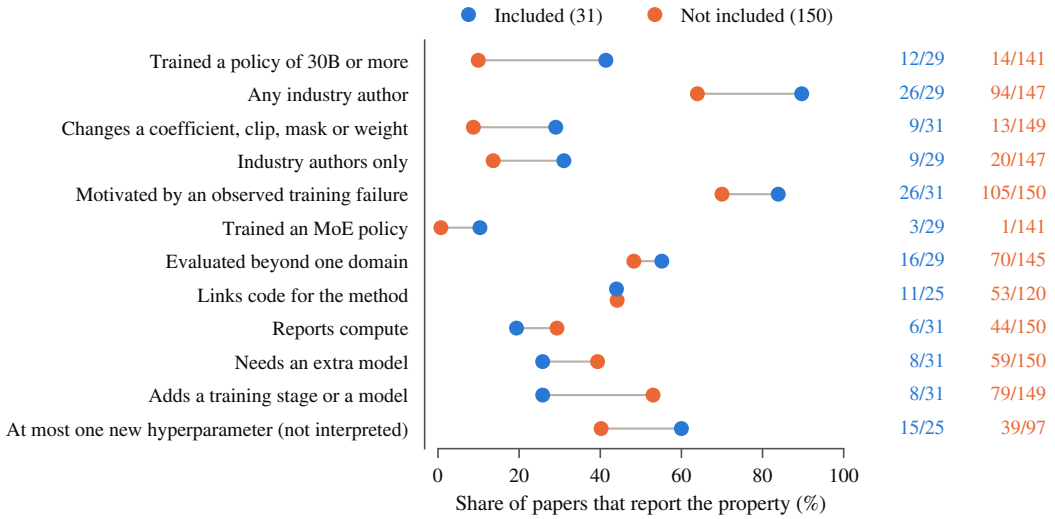


Figure 3: Coded fields, included against non-included general methods from 2025. Unit: paper. Each row compares the share among included papers with the share among papers from the same quarters that no framework included. The right columns give the counts, included first. Two shares within three points of each other are drawn one above the other. Papers that do not report a property are left out of that row.

Scale shows the largest gap. Policies of 30B parameters or more appear in 41 percent of included papers and 10 percent of the others. The gap persists after excluding model reports, at 12 of 29 against 13 of 139, and within papers that have an industry author, at 10 of 25 against 11 of 90. Counting every unreported size against the gap still leaves 12 of 31 against 23 of 150.

The form of the change also differs. A coefficient, clip, mask or weight in the existing loss was the change in 29 percent of included papers and 9 percent of the others, while a new training stage or model was the change in 26 against 53 percent. The frameworks expose the widely included methods in the same compact form: one or two configuration fields or loss names, such as `clip_ratio_high` in verl and `epsilon_high` in TRL for clip-higher (Table 11). The compact form describes the method and not the size of its code, because pull requests for coefficient changes were about as large as those for new loss terms (Section 7). The number of new hyperparameters is missing for many papers, and under the least favourable treatment of missing values its gap reverses, so we do not interpret it.

The two gaps hold outside the frameworks' own companies. Included papers come from a company that maintains one of the frameworks more often than others, at 32 against 15 percent, but only 5 were included by that company's framework. Across all 37 included papers, 11 come from such a company and 6 were included by its own framework, such as DAPO in verl and DLER in NeMo-RL. Inclusion also crosses company lines through the people who commit: all 18 papers with an outside committer had such a landing in a framework outside their own company. Without any such paper in either group, large policies appear in 6 of 19 included papers against 9 of 119 others, and coefficient changes in 6 of 21 against 13 of 128 (Table 7).

Code release was equally common in both groups. The first version links training code in 44 percent of included papers and 44 percent of others, so a public implementation alone does not set the included methods apart. Other fields point the same way as the main gaps with smaller differences. A named training failure motivated 84 percent of included papers and 70 percent of others, and evaluation beyond one domain appears in 55 against 48 percent of them. Industry authors appear on 90 percent of included papers and 64 percent of others, while included papers reported compute less often, at 19 against 29 percent.

Inclusion also differs by component (Figure 4). Methods that change clipping or the trust region, off-policy correction, or loss aggregation make up 187 of the general methods, and 10 of these reached a framework, 3 of them three or more. Rollout sampling is the most crowded component. Together with credit assignment it accounts for 554 general methods, 4 inclusions and none in three or more frameworks. This comparison covers framework options only (Section 11).

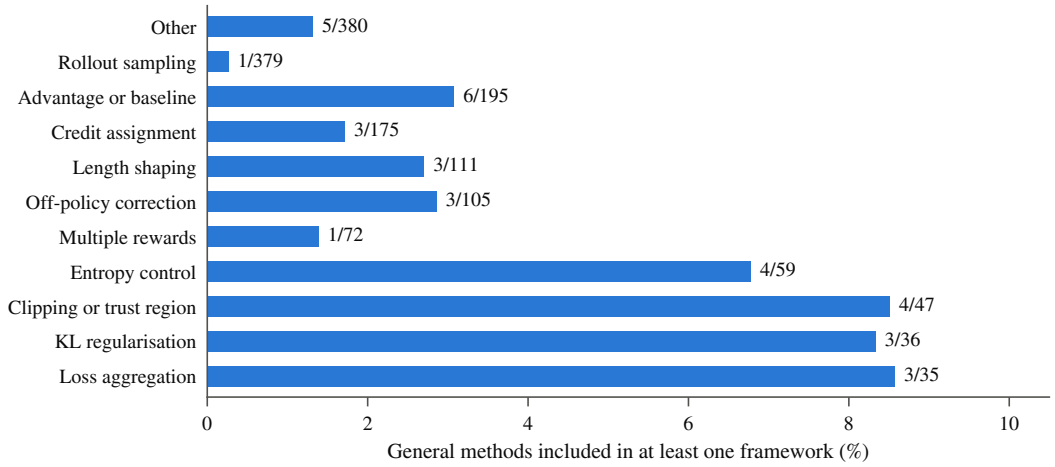


Figure 4: Inclusion by component. Unit: general method, each with one primary component label. Share of general methods in each component that reached at least one framework, with counts. Rows are ordered by the number of general methods, and components with fewer than ten are omitted.

Most of the widely included methods are motivated by a failure that appears in long or large runs: entropy collapse, length bias, extreme importance ratios, instability in mixture-of-experts models, and the mismatch between rollout and training probabilities (Table 1).

7 Proposals and merges

Table 2: Methods and proposals by framework. Unit: landing of a catalog method with a source from 2025 or later, and proposals for general methods that no framework included. Methods counts the landings in the framework, and first anywhere counts the methods with their first landing there. Median days run from the source to the landing and leave out upper bounds. A landing pull request counts as opened by a prior merger when its author had merged a pull request in the framework before opening it, and each pull request counts once. Days open run from opening to merge. Proposals are the coded issues and pull requests of Table 9, and lapsed proposals were closed as inactive or in a stale-issue cleanup. Declined proposals include those redirected elsewhere. ROLL publishes code in bulk releases without pull requests.

		First	Median	Landing pull requests		Proposals for other methods			
Framework	Methods	anywhere	days	By a prior merger	Days open	All	Open	Declined	Lapsed
verl	30	21	25	1 of 27	10	20	12	0	7
TRL	24	16	35	13 of 23	5	18	3	10	2
AReaL	14	4	170	3 of 13	6	1	0	0	1
NeMo-RL	14	3	167	2 of 13	17	2	2	0	0
ROLL	12	4	99	–	–	0	0	0	0
slime	11	3	64	6 of 7	0	4	2	2	0
OpenRLHF	9	3	34	4 of 5	0	3	3	0	0

The features above describe the methods that got in. The public records of the frameworks show how the others fared, and the main finding is an absence: maintainers turned away few methods, 12 of 1,561, because few were ever proposed. We downloaded all 26,960 issues, pull requests and discussions of the seven frameworks on 18 October 2026 (UTC) and searched them for every general method in the pool. A paper has a record when a thread links its arXiv page, gives its arXiv identifier or quotes its full title.

Few methods appear in a framework's public record. Most general methods that no framework included have no record in any of the seven frameworks, 1,500 of 1,561, and another 21 are cited only in passing, for example in lists of projects built on a framework. For 25 papers someone opened a pull request to implement the method, and for 15 more someone requested it in an issue or a discussion.

Chat groups and internal repositories leave no public trace, so a missing record does not show that maintainers never saw a paper.

Method names were checked separately, because many are ambiguous. Among the 1,500 general methods without a record, 726 have a method name that occurs once in the pool, and 23 of these names appear in the threads. Reading those threads showed that 13 refer to the paper, 8 as proposals and 5 in passing. The other matches share a name with something else, such as the NCCL SHARP plugin. Counting the 13 lowers the share without a record from 96 to 95 percent, and applying the same rate to papers without a searchable name gives 94 percent. The coded proposals below come from link, identifier and title matches, so the 40 papers with a proposal are a lower bound: the name search found 8 more.

Many proposals stay open without a reply. We coded each of the 48 threads on these 40 papers from its own text (Table 9). By the snapshot, 17 of the 28 pull requests were still open after a median of 196 days, and no prior merger had replied in 13 of them. Maintainers declined 4 pull requests and redirected 2, while the rest were withdrawn, closed as inactive or closed without a comment. Requests fared similarly: of the 20 in issues and discussions, a scheduled cleanup of inactive issues closed 8, and maintainers declined 4 and redirected 2. In 16 of the 17 open threads without a prior-merger reply, nobody except the proposer and automated reviewers had replied.

Merged pull requests drew concerns too. One reader coded the 88 merged landing pull requests and the 48 proposals with the same fields, each from the thread's own text (Table 3). Someone other than the proposer raised a concern in 60 merged pull requests, mostly about design or missing tests and documentation. The proposer revised the code in 49 of the 88, and 41 still had an open concern when they were merged. Prior mergers raised the concerns in 57 of the 60 merged threads with a concern and in 17 of the 18 other threads with one.

Table 3: Review records of merged pull requests and of proposals that were not merged. Unit: thread. Merged pull requests are the landing pull requests of recent methods, counted once each. Unmerged pull requests and requests are the proposals of Table 9. A concern is a problem that someone other than the proposer raises explicitly, and one thread can carry several.

Thread record	Merged pull requests	Unmerged pull requests	Requests
Threads	88	28	20
Reply from someone other than the proposer	71	14	10
Concern raised in the thread	60	11	7
design or interface	50	7	1
tests or documentation	39	4	0
evidence	18	3	1
placement	8	4	4
maintenance burden	6	5	6
thin demand	0	3	5
already covered	5	2	3
Concern still open at the end	41	9	7
Training results shown	26	9	2
Results requested	17	3	1
Demand stated in the thread	14	3	9

Concerns about demand and maintenance burden follow a different pattern: they appear in 6 of 88 merged pull requests, against 5 of 28 unmerged pull requests and 6 of 20 requests. All of these unmerged pull requests come from TRL, so this pattern describes TRL more than the whole ecosystem. Training results appear about as often in merged and unmerged pull requests, at 26 of 88 and 9 of 28. The largest difference is attention: 14 of the 28 unmerged pull requests drew no reply from anyone but the proposer, against 17 of the 88 merged ones. TRL weighs demand by the number of requests and reactions, and in one thread by whether other frameworks implement the method. slime keeps its main repository for changes that CI or routine training runs can verify, and it points other methods to separate repositories with reproducible recipes.

The frameworks handle proposals in different ways (Table 2). In TRL, prior mergers opened 13 of the 23 pull requests that brought methods in, and the project declined or redirected 10 of 18 proposals for

other methods. verl takes methods in from other contributors, often as recipes: prior mergers opened 1 of its 27 landing pull requests, while 12 of 20 proposals for other methods remain open and 7 lapsed. slime and OpenRLHF add methods mainly through prior mergers, at 6 of 7 and 4 of 5 landing pull requests. NeMo-RL reviewers asked for training results in 8 of its 13 merged pull requests.

Some pull requests brought in several methods, so the 94 landings of recent methods with a pull request came from 88 distinct pull requests. Paper authors made a commit in 21 of the landings, and 22 followed an earlier request in the same repository. Counting each pull request once, prior mergers opened 29 of the 88. Merged pull requests had been open for a median of 6.5 days, a figure that covers only proposals that were merged. Inside each framework's own package they changed a median of 117 lines, against 162 for proposals that were not merged. The form of the change coded from the paper relates only weakly to this size: medians are 88 lines for coefficient changes, 80 for new terms and 141 for pipeline changes, with each pull request counted once per form it carries.

Inclusion rarely reverses: 4 of the 114 landings of recent methods were later removed. TRL dropped the experimental trainers for GFPO, DPPO and PAPO, and AReaL dropped the AEnt recipe in a refactor. NeMo-RL reverted its first GSPO commit a day after it landed and added the method again with test fixes four days later.

One method arrived as a fix. In June 2025 a NeMo-RL developer implemented the KL gradient correction of Tang and Munos [69] behind a switch and withdrew it, because the experiments showed no gain. A year later a fix to the KL backward pass made the corrected gradient the behaviour of every run with a KL penalty, and its code cites the paper. We count this fix as the method's landing, as a conditional default.

8 Failures that users report

The frameworks' issues are where users would report the failures that the catalog's methods address, so they record what users name in public. We screened the 7,075 issues and discussions opened in the seven frameworks between January 2025 and the snapshot with keyword patterns for the thirteen failures of the catalog, and one reader coded the title and opening post of each of the 648 matches (Appendix E). A thread counts when its opener reports observing the failure in a run. Most matches were bugs, usage questions or feature requests, and 48 threads report a training failure (Table 4). Among them, 23 describe a collapse or divergence with no cause named, such as a loss that turns to NaN or a reward that drops to zero, and 12 name the mismatch between rollout and training probabilities. A random sample of 250 unscreened threads, read the same way, contained no failure report.

The failures named in public differ from those that papers address. The failures that abstracts address most often, sparse reward in 218 general methods and the loss of diversity in 156, appear in no screened thread. The mismatch between rollout and training probabilities is named by 12 threads, and 20 general methods and 8 catalog methods in the frameworks address it, more than for any other failure. The record measures what users name in public and not what they need, because an abstract and an issue serve different purposes and the 23 collapses with no cause named may belong to any row of the table. Requests for methods outnumber failure reports: 59 threads ask for one, 8 of them for DAPO's dynamic sampling, 6 for TIS or MIS and 4 for rollout routing replay, all methods that several frameworks already include. TRL weighs demand by such requests (Section 7), and these counts show how thin that signal is: the most requested method drew 8 requests across seven frameworks in 21 months. The random sample of 250 unscreened threads supports the screen and no more, so the counts are lower bounds, and a count of zero shows that nobody named the failure and not that nobody met it.

9 Methods in large model reports

Inclusion describes framework code, so we read the technical reports of large open-weight models for the included methods their training used. The pool holds 13 reports of open-weight models with 100B parameters or more, from DeepSeek-R1 in January 2025 to Ling and Ring 2.6 in June 2026 (Table 5). The pool requires reinforcement learning in the title or abstract, so reports that mention RL only in the body fall outside it: Qwen3 [81], the model behind GSPO, is one of them, and Appendix E lists the others we know of. For each report we read the RL sections and recorded the training algorithm,

Table 4: Training failures in the frameworks' issues against the methods that address them. Unit: thread in the first column, general method in the next two, catalog method in the last. Issue threads are issues and discussions opened since January 2025 with an opening post that reports the failure. General methods naming the failure are the pool's general methods with an abstract that names it, with the number of those that at least one framework includes. Catalog methods are those addressing the failure that at least one framework includes. The two rows with dashes are failures outside the catalog's list.

	Issue threads	General methods		Catalog methods
Failure	naming it	naming it	included	in a framework
Collapse or divergence, cause not named	23	–	–	–
Rollout-training mismatch	12	20	3	8
Response-length drift	5	167	5	5
Stale rollout data	3	52	4	7
KL penalty grows	3	–	–	–
Groups without signal	1	109	2	2
MoE instability	1	6	4	3
Sparse reward	0	218	4	3
Loss of diversity	0	156	2	3
Gradient variance	0	115	8	6
Entropy collapse	0	87	4	5
Value-model cost	0	31	6	6
Extreme importance ratios	0	18	5	7
Multi-reward collapse	0	14	1	1
Truncation noise	0	5	0	1

the framework it names and the widely included methods of Table 1 it states that it used. A method counts as stated when the report names it or cites its source, and a change described under another name counts as a variant.

Table 5: Widely included methods stated by technical reports of open-weight models with 100B parameters or more. Unit: report in the pool. A check marks a method the report states it used, S marks the report that is the method's source, V a variant described under another name, and T a method tried and not kept. Framework stated is the training system the report names.

Report	Size	Framework stated	Clip-higher	GSPO	Dyn. sampling	Dr. GRPO	CISPO	TIS	MIS	R++	Token loss	Overlong	R3	SAPO	GDPO
DeepSeek-R1 [13]	671B MoE	in-house													
Llama-Nemotron [4]	253B dense	NeMo-Aligner													
MiniMax-M1 [49]	456B MoE	not stated	T		✓		S				✓	✓			
Kimi K2 [32]	1T MoE	in-house													
gpt-oss [53]	117B MoE	in-house													
GLM-4.5 [22]	355B MoE	slime			✓						✓				
LongCat-Flash-Thinking [47]	560B MoE	DORA (in-house)	✓		V	✓					✓	V			
Ring-1T [37]	1T MoE	AReaL						T	S						
DeepSeek-V3.2 [14]	671B MoE	in-house											V		
LongCat-Flash-Thinking-2601 [48]	560B MoE	DORA (in-house)		✓											
MiniMax-M2 [8]	230B MoE	in-house					✓								
Nemotron 3 Ultra [52]	550B MoE	NeMo-RL							✓						
Ling and Ring 2.6 [33]	1T MoE	AReaL							V						

The reports name the included methods, but each names few. Of the 13 reports, 6 state at least one of the 13 methods, 2 are the source of one, and 3 describe a variant under their own name. DeepSeek's Keep Routing replays the rollout's expert routes as rollout routing replay does, LongCat filters all-correct and all-wrong prompts with replacement where DAPO samples without, and KPop replaces IcePop's ratio bounds with a binary KL test. Token-level loss aggregation is stated most often, by 3 reports. The KL term is removed in 4 reports, a change every framework allows through its KL coefficient and TRL made the default. None of the methods appears in 4 reports: DeepSeek-R1

and Llama-Nemotron predate most of them, Kimi K2 keeps the K1.5 objective, and gpt-oss gives no detail. No report states SAPO, GDPO or REINFORCE++, rollout routing replay appears only as a variant, and TIS appears only as a comparison. One of the seven frameworks is named in 4 reports: GLM-4.5 trains on slime, Ring-1T and Ling and Ring 2.6 on AReaL, and Nemotron 3 Ultra on NeMo-RL. Llama-Nemotron names NeMo-Aligner, the predecessor of NeMo-RL, and the other reports train on in-house systems. The reports are a second observable record of practice, and it overlaps with the frameworks' code: the stabilisers that six or seven frameworks ship are the ones the reports state. The overlap does not show the order of influence, because we read the latest version of each report, and the frameworks took in GSPO and MIS within weeks of the papers that introduced them.

10 Implications for researchers and maintainers

The comparison and the record describe past inclusion, and each paragraph below pairs a property of included methods with one step that a paper and its authors can take. None of these steps is shown to raise the chance of inclusion.

A public proposal. Most methods never enter a framework's public record, while 22 of the 94 landings followed an earlier issue or discussion that cited the paper. A record does not secure attention, since verl closed 7 of its 20 proposals for other methods in scheduled cleanups. A short issue that names the training failure, the change and the evidence still gives maintainers something to find and answer.

Framework rules. Each framework states its own rules. TRL asks for tests, documentation and an entry in its paper index, and it adds options to its main trainer only with visible demand. slime accepts changes that routine runs can verify, and verl collects many methods as recipes. General libraries can set stricter thresholds: the rule of thumb of scikit-learn for a new algorithm is three years since publication, 200 citations and wide use [60]. The RL frameworks took methods in within a median of 30 days, so their written rules and their review do the selecting. Merged pull requests drew concerns about design and tests in most cases, and their proposers often revised the code. A proposal that follows the written rules answers part of these concerns in advance.

Results that maintainers can rerun. Training results in the thread did not separate merged from unmerged pull requests, but NeMo-RL reviewers asked for them in 8 of 13 merged pull requests, and TRL asked for reproduced results before adding a method to its main trainer. A configuration, a seed and a short run on a public model let maintainers check that the method runs and shows its signal. Matching the paper's gains takes its baselines, data and budget.

The failure and its signal. Included papers more often named an observed training failure, at 84 against 70 percent. Reporting the signal of that failure, such as the entropy curve or the divergence between rollout and training probabilities, lets others check the fix on their own runs.

Evidence at scale. Included papers more often trained policies of 30B parameters or more. Teams without that compute can still target failures that larger runs report and publish the signal above, so that groups with more compute can test the fix.

Changes stated as configuration fields. Included papers more often changed a coefficient, clip, mask or weight in the existing loss, and frameworks expose these methods as one or two configuration fields. A paper can name the configuration field it changes, the training stage it touches and any models it needs, so that maintainers can place the change without reading the code.

Choice of component. Frameworks included 10 of 187 general methods on clipping, off-policy correction and loss aggregation, and 4 of 554 on rollout sampling and credit assignment. Methods in the second group can also live in user code, outside the options we track. In the frameworks' issues, the mismatch between rollout and training probabilities was named in 12 failure reports and 6 method requests, while no screened thread named sparse reward or the loss of diversity (Section 8).

For maintainers. The record also shows what a framework could change, under the same caveat. Proposals go unanswered more often than they are refused: 13 of the 17 open pull requests for other methods had no reply from a prior merger after a median of 196 days. Scheduled cleanups closed 8 of 20 requests and 7 of 20 proposals in verl. A stated route for methods, such as verl's recipes or slime's separate repositories, lets a maintainer answer a proposal with a destination instead of leaving it to the cleanup. A proposal template that asks for the failure, the change, the evidence and the configuration field would put in the first message the items that reviewers raised in 60 of 88 merged pull requests. Demand measured by requests is thin (Section 8), so a framework that waits for requests before adding an option will see few. The technical reports of large runs (Section 9) name the failures those runs met and the methods they used, a second public record a framework can read.

11 Limitations

Inclusion is not use: we observe implementations on main branches, and the data do not show the frequency of use or the results users get. Section 9 reads what 13 reports state, and a statement in a report is not code either. The numerator is a lower bound, because it counts only options we can parse and implementations that framework files cite. Sampling and reward logic can live outside framework options, so the component comparison covers framework options only. The pool is screened from three arXiv categories, and the seven frameworks represent neither closed training stacks nor every open one. Four other open frameworks meet the selection criteria, and their current code adds 2 general methods to the included set (Section 3).

The comparison in Section 6 is observational and small. Affiliations come from each paper's first page and match a framework at the level of the company, so a team that works with a framework's maintainers from another company counts as outside. The public record misses chat groups and internal repositories, and the search by method name reaches only names that are distinctive. ROLL and AReaL publish much of their code in bulk releases. Landings without a pull request still went through review, and the pull requests with a record do not represent each framework's whole process. Thread texts can be edited or deleted after the fact, and one reader coded the review fields of all threads. The failure reading of Section 8 covers titles and opening posts after a keyword screen, by one reader, so its counts are lower bounds, and a collapse reported without a cause may belong to any of the failures. The reports of Section 9 cover 13 models from nine organisations and state what their authors chose to write. Whether maintainers act on such reports, on requests or on their own runs is not measured here, so the overlap between the reports and the frameworks' code is an observation and not a direction of influence.

12 The tracker

The data and code behind this report form a maintained tracker at `https://github.com/pth2002/rl-method-tracker`. Its index groups the catalog by the training failure each method addresses. For every method it shows the frameworks that include it, the tier and the day it landed, with the setting that selects the method and a link to the file and line at the analysed commit. A second page gives, for each framework, its contribution rules, the files that hold its algorithms, the pull requests that brought recent methods in, the concerns raised in their review and the outcome of earlier proposals. Readers can report a wrong record through a correction form in the repository. A weekly job pulls the frameworks, re-traces their history, verifies sources against arXiv and opens a pull request. An option the parser detects without a mapping fails validation, so a person reviews every addition before it reaches the index. The release fixes the samples, labels, full-text coding and framework commits behind this report, together with a checksum list. It also stores the record of proposals. For each general method it lists the issues and pull requests that cite it, their state at the snapshot and the coded outcome. Thread texts stay on GitHub, and each coded thread keeps one short quote as evidence for its code. The release also stores the coded failure reports of Section 8 and the readings of the model reports of Section 9, each with one quote. The weekly job also refreshes this record and lists new proposals for coding. A separate command checks the inputs against that list and reproduces the report's numbers, tables and figures without network access.

13 Conclusion

Most RL methods for language models reach practitioners as papers and not as options in the frameworks they train with. Between January 2025 and June 2026, 2.3 percent of the general RL training methods in our pool reached one of seven open-source frameworks, and three or more frameworks include 7 of 1,598 today. Across five quarterly cohorts the papers grew 5.4-fold while those included within 180 days stayed at 5 to 7. Included methods more often changed one part of the existing loss and more often came with experiments on large policies, while released code was as common among them as among the others. Merged pull requests drew concerns about design and tests as well, while concerns about demand and maintenance burden appeared mostly in proposals that were not merged. Yet few methods were turned away in public, because most never reached a framework's issues or pull requests at all. Users rarely report training failures in those issues, and the technical reports of large open-weight models state the methods that most frameworks include, so the public record of demand is thin and the record of practice at scale overlaps with the frameworks' code. Other fields with shared training or evaluation code can reuse this measurement, since it needs only paper versions, implementation histories and public proposals. The tracker lists where each framework keeps its algorithms, the concerns its reviewers raised and the outcome of earlier proposals, so that authors can bring a method that far.

References

[1] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE style optimization for learning from human feedback in LLMs. *arXiv preprint arXiv:2402.14740*, 2024.

[2] Nur Ahmed, Muntasir Wahed, and Neil C. Thompson. The growing influence of industry in AI research. *Science*, 379(6635):884–886, 2023.

[3] Lei Bai, Zhongrui Cai, Yuhang Cao, Maosong Cao, Weihan Cao, Chiyu Chen, Haojiong Chen, Kai Chen, Pengcheng Chen, Ying Chen, et al. Intern-S1: A scientific multimodal foundation model. *arXiv preprint arXiv:2508.15763*, 2025.

[4] Akhiad Bercovich, Itay Levy, Izik Golan, Mohammad Dabbah, Ran El-Yaniv, Omri Puny, Ido Galil, Zach Moshe, Tomer Ronen, Najeeb Nabwani, et al. Llama-Nemotron: Efficient reasoning models. *arXiv preprint arXiv:2505.00949*, 2025.

[5] Aaditya Bhatia, Ellis E. Eghan, Manel Grichi, William G. Cavanagh, Zhen Ming, Jiang, and Bram Adams. Towards a change taxonomy for machine learning systems. *arXiv preprint arXiv:2203.11365*, 2022.

[6] Adam Block and Cyril Zhang. EMA without the lag: Bias-Corrected iterate averaging schemes. *arXiv preprint arXiv:2508.00180*, 2025.

[7] Kianté Brantley, Mingyu Chen, Zhaolin Gao, Jason D. Lee, Wen Sun, Wenhao Zhan, and Xuezhou Zhang. Accelerating RL for LLM reasoning with optimal advantage regression. *arXiv preprint arXiv:2505.20686*, 2025.

[8] Aili Chen, Aonian Li, Baichuan Zhou, Bangwei Gong, Binyang Jiang, Boji Dan, Changhao Zhang, Changqing Yu, Chao Wang, Cheng Ma, et al. The MiniMax-M2 series: Mini activations unleashing max Real-World intelligence. *arXiv preprint arXiv:2605.26494*, 2026.

[9] Xiangxiang Chu, Hailang Huang, Xiao Zhang, Fei Wei, and Yong Wang. GPG: A simple and strong reinforcement learning baseline for model reasoning. *arXiv preprint arXiv:2504.02546*, 2025.

[10] Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, et al. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025.

[11] Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.

[12] Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. Are we really making much progress? a worrying analysis of recent neural recommendation approaches. *arXiv preprint arXiv:1907.06902*, 2019.

[13] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[14] DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, et al. DeepSeek-V3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.

[15] Mostafa Dehghani, Yi Tay, Alexey A. Gritsenko, Zhe Zhao, Neil Houlsby, Fernando Diaz, Donald Metzler, and Oriol Vinyals. The benchmark lottery. *arXiv preprint arXiv:2107.07002*, 2021.

[16] Yuyang Ding, Chi Zhang, Juntao Li, Haibin Lin, and Min Zhang. FAPO: Flawed-Aware policy optimization for efficient and reliable reasoning. *arXiv preprint arXiv:2510.22543*, 2025.

[17] Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep policy gradients: A case study on PPO and TRPO. *arXiv preprint arXiv:2005.12729*, 2020.

[18] Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-Group policy optimization for LLM agent training. *arXiv preprint arXiv:2505.10978*, 2025.

[19] Denae Ford, Mahnaz Behroozi, Alexander Serebrenik, and Chris Parnin. Beyond the code itself: How programmers really look at pull requests. In *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS)*, pages 51–60, 2019.

[20] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. AReaL: A Large-Scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.

[21] Chang Gao, Chujie Zheng, Xiong-Hui Chen, Kai Dang, Shixuan Liu, Bowen Yu, An Yang, Shuai Bai, Jingren Zhou, and Junyang Lin. Soft adaptive policy optimization. *arXiv preprint arXiv:2511.20347*, 2025.

[22] GLM-4. 5 Team, Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, et al. GLM-4.5: Agentic, reasoning, and coding (ARC) foundation models. *arXiv preprint arXiv:2508.06471*, 2025.

[23] Georgios Gousios, Martin Pinzger, and Arie van Deursen. An exploratory study of the pull-based software development model. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, pages 345–355, 2014.

[24] Georgios Gousios, Andy Zaidman, Margaret-Anne Storey, and Arie van Deursen. Work practices and challenges in pull-based development: The integrator's perspective. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*, 2015.

[25] Yaru Hao, Li Dong, Xun Wu, Shaohan Huang, Zewen Chi, and Furu Wei. On-Policy RL with optimal reward baseline. *arXiv preprint arXiv:2505.23585*, 2025.

[26] Jujie He, Jiacai Liu, Chris Yuhao Liu, Rui Yan, Chaojie Wang, Peng Cheng, Xiaoyu Zhang, Fuxiang Zhang, Jiacheng Xu, Wei Shen, et al. Skywork open reasoner 1 technical report. *arXiv preprint arXiv:2505.22312*, 2025.

[27] Jian Hu, Xibin Wu, Wei Shen, Jason Klein Liu, Zilin Zhu, Weixun Wang, Songlin Jiang, Haoran Wang, Hao Chen, Bin Chen, et al. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. *arXiv preprint arXiv:2405.11143*, 2024.

[28] Jian Hu, Jason Klein Liu, Haotian Xu, and Wei Shen. REINFORCE++: Stabilizing Critic-Free policy optimization with global advantage normalization. *arXiv preprint arXiv:2501.03262*, 2025.

[29] Shengyi Huang, Rousslan Fernand Julien Dossa, Antonin Raffin, Anssi Kanervisto, and Weixun Wang. The 37 implementation details of proximal policy optimization. ICLR Blog Track, 2022.

[30] Bi Huo, Bin Tu, Cheng Qin, Da Zheng, Debing Zhang, Dongjie Zhang, En Li, Fu Guo, Jian Yao, Jie Lou, et al. dots.llm1 technical report. *arXiv preprint arXiv:2506.05767*, 2025.

[31] Devvrit Khatri, Lovish Madaan, Rishabh Tiwari, Rachit Bansal, Sai Surya Duvvuri, Manzil Zaheer, Inderjit S. Dhillon, David Brandfonbrener, and Rishabh Agarwal. The art of scaling reinforcement learning compute for LLMs. *arXiv preprint arXiv:2510.13786*, 2025.

[32] Kimi Team, Yifan Bai, Yiping Bao, Y. Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, et al. Kimi K2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.

[33] Ang Li, Ben Liu, Bin Han, Bin Hu, Bin Jing, Binbin Hu, Bing Li, Cai Chen, Caizhi Tang, Changxin Tian, et al. Ling and ring 2.6 technical report: Efficient and instant agentic intelligence at Trillion-Parameter scale. *arXiv preprint arXiv:2606.15079*, 2026.

[34] Yingru Li, Jiacai Liu, Jiawei Xu, Yuxuan Tong, Ziniu Li, Qian Liu, and Baoxiang Wang. Trust region masking for Long-Horizon LLM reinforcement learning. *arXiv preprint arXiv:2512.23075*, 2025.

[35] Yingru Li, Jiawei Xu, Ziniu Li, Jiacai Liu, Wei Liu, Yuxuan Tong, Longtao Zheng, Zhenghai Xue, Yaxiang Zhang, Tianle Cai, et al. The optimal token baseline: Variance reduction for Long-Horizon LLM-RL. *arXiv preprint arXiv:2602.07078*, 2026.

[36] Ziniu Li, Tian Xu, Yushun Zhang, Zhihang Lin, Yang Yu, Ruoyu Sun, and Zhi-Quan Luo. ReMax: A simple, effective, and efficient reinforcement learning method for aligning large language models. *arXiv preprint arXiv:2310.10505*, 2023.

[37] Ling Team, Anqi Shen, Baihui Li, Bin Hu, Bin Jing, Cai Chen, Chao Huang, Chao Zhang, Chaokun Yang, Cheng Lin, et al. Every step evolves: Scaling reinforcement learning for Trillion-Scale thinking model. *arXiv preprint arXiv:2510.18855*, 2025.

[38] Zachary C. Lipton and Jacob Steinhardt. Troubling trends in machine learning scholarship. *arXiv preprint arXiv:1807.03341*, 2018.

[39] Fanfan Liu, Youyang Yin, Peng Shi, Siqi Yang, Zhixiong Zeng, and Haibo Qiu. Length-Unbiased sequence policy optimization: Revealing and controlling response length variation in RLVR. *arXiv preprint arXiv:2602.05261*, 2026.

[40] Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, Yejin Choi, et al. DLER: Doing length penalty right - incentivizing more intelligence per token via reinforcement learning. *arXiv preprint arXiv:2510.15110*, 2025.

[41] Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Peter Belcak, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, et al. GDPO: Group reward-decoupled normalization policy optimization for multi-reward RL optimization. *arXiv preprint arXiv:2601.05242*, 2026.

[42] Yuhang Liu, Zeyu Liu, Shuanghe Zhu, Pengxiang Li, Congkai Xie, Jiasheng Wang, Xavier Hu, Xiaotian Han, Jianbo Yuan, Xinyao Wang, et al. InfiGUI-G1: Advancing GUI grounding with adaptive exploration policy optimization. *arXiv preprint arXiv:2508.05731*, 2025.

[43] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-Like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.

[44] Zihe Liu, Jiashun Liu, Yancheng He, Weixun Wang, Jiaheng Liu, Ling Pan, Xinyu Hu, Shaopan Xiong, Ju Huang, Jian Hu, et al. Part i: Tricks or traps? a deep dive into RL for LLM reasoning. *arXiv preprint arXiv:2508.08221*, 2025.

[45] Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, and Fuli Luo. Stabilizing MoE reinforcement learning by aligning training and inference routers. *arXiv preprint arXiv:2510.11370*, 2025.

[46] Willem Meijer, Mirela Riveni, and Ayushi Rastogi. Ecosystem-wide influences on pull request decisions: Insights from NPM. *Empirical Software Engineering*, 30(6), 2025.

[47] Meituan LongCat Team, Anchun Gui, Bei Li, Bingyang Tao, Bole Zhou, Borun Chen, Chao Zhang, Chao Zhang, Chengcheng Han, Chenhui Yang, et al. Introducing LongCat-Flash-Thinking: A technical report. *arXiv preprint arXiv:2509.18883*, 2025.

[48] Meituan LongCat Team, Anchun Gui, Bei Li, Bingyang Tao, Bole Zhou, Borun Chen, Chao Zhang, Chao Zhang, Chen Gao, Chen Zhang, et al. LongCat-Flash-Thinking-2601 technical report. *arXiv preprint arXiv:2601.16725*, 2026.

[49] MiniMax, Aili Chen, Aonian Li, Bangwei Gong, Binyang Jiang, Bo Fei, Bo Yang, Boji Shan, Changqing Yu, Chao Wang, et al. MiniMax-M1: Scaling Test-Time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025.

[50] Kevin Musgrave, Serge Belongie, and Ser-Nam Lim. A metric learning reality check. *arXiv preprint arXiv:2003.08505*, 2020.

[51] NeMo RL. NeMo RL: A scalable and efficient post-training library. `https://github.com/NVIDIA-NeMo/RL`, 2025.

[52] NVIDIA, Aaron Blakeman, Aaron Thomas, Aastha Jhunjhunwala, Abhibha Gupta, Abhinav Khattar, Adam Rajfer, Adi Renduchintala, Adil Asif, Aditya Vavre, et al. Nemotron 3 ultra: Open, efficient Mixture-of-Experts hybrid Mamba-Transformer model for agentic reasoning. *arXiv preprint arXiv:2606.15007*, 2026.

[53] OpenAI, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.

[54] Prime Intellect Team, Sami Jaghouar, Justus Mattern, Jack Min Ong, Jannik Straube, Manveer Basra, Aaron Pazdera, Kushal Thaman, Matthew Di Ferrante, Felix Gabriel, et al. INTELLECT-2: A reasoning model trained through globally decentralized reinforcement learning. *arXiv preprint arXiv:2505.07291*, 2025.

[55] Penghui Qi, Xiangxin Zhou, Zichen Liu, Tianyu Pang, Chao Du, Min Lin, and Wee Sun Lee. Rethinking the trust region in LLM reinforcement learning. *arXiv preprint arXiv:2602.04879*, 2026.

[56] Pierre Harvey Richemond, Yunhao Tang, Daniel Guo, Daniele Calandriello, Mohammad Gheshlaghi Azar, Rafael Rafailov, Bernardo Avila Pires, Eugene Tarassov, Lucas Spangher, Will Ellsworth, et al. Offline regularised reinforcement learning for large language models alignment. *arXiv preprint arXiv:2405.19107*, 2024.

[57] Nicolas Le Roux, Marc G. Bellemare, Jonathan Lebensold, Arnaud Bergeron, Joshua Greaves, Alex Fréchette, Carolyne Pelletier, Eric Thibodeau-Laufer, Sándor Toth, and Sam Work. Tapered Off-Policy REINFORCE: Stable and efficient reinforcement learning for LLMs. *arXiv preprint arXiv:2503.14286*, 2025.

[58] John Schulman. Approximating KL divergence. `http://joschu.net/blog/kl-approx.html`, 2020.

[59] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[60] scikit-learn. scikit-learn FAQ: What are the inclusion criteria for new algorithms? `https://scikit-learn.org/stable/faq.html`.

[61] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[62] Guobin Shen, Chenxiao Zhao, Xiang Cheng, Lei Huang, and Xing Yu. VESPO: Variational Sequence-Level soft policy optimization for stable Off-Policy LLM training. *arXiv preprint arXiv:2602.10693*, 2026.

[63] Han Shen. On entropy control in LLM-RL algorithms. *arXiv preprint arXiv:2509.03493*, 2025.

[64] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. *arXiv preprint arXiv:2409.19256*, 2024.

[65] Vaishnavi Shrivastava, Ahmed Awadallah, Vidhisha Balachandran, Shivam Garg, Harkirat Behl, and Dimitris Papailiopoulos. Sample more to think less: Group filtered policy optimization for concise reasoning. *arXiv preprint arXiv:2508.09726*, 2025.

[66] Vaishnavi Shrivastava, Piero Kauffmann, Ahmed Awadallah, and Dimitris Papailiopoulos. ECHO: Terminal agents learn world models for free. *arXiv preprint arXiv:2605.24517*, 2026.

[67] StepFun, Bin Wang, Bojun Wang, Changyi Wan, Guanzhe Huang, Hanpeng Hu, Haonan Jia, Hao Nie, Mingliang Li, Nuo Chen, et al. Step-3 is large yet affordable: Model-system co-design for cost-effective decoding. *arXiv preprint arXiv:2507.19427*, 2025.

[68] Yehui Tang, Yichun Yin, Yaoyuan Wang, Hang Zhou, Yu Pan, Wei Guo, Ziyang Zhang, Miao Rang, Fangcheng Liu, Naifu Zhang, et al. Pangu ultra MoE: How to train your big MoE on ascend NPUs. *arXiv preprint arXiv:2505.04519*, 2025.

[69] Yunhao Tang and Rémi Munos. On a few pitfalls in KL divergence gradient estimation for RL. *arXiv preprint arXiv:2506.09477*, 2025.

[70] Yunhao Tang, Kunhao Zheng, Gabriel Synnaeve, and Rémi Munos. Optimizing language models for inference time objectives using reinforcement learning. *arXiv preprint arXiv:2503.19595*, 2025.

[71] Team Cohere, Aakanksha, Arash Ahmadian, Marwan Ahmed, Jay Alammar, Milad Alizadeh, Yazeed Alnumay, Sophia Althammer, Arkady Arkhangorodsky, Viraat Aryabumi, et al. Command a: An Enterprise-Ready large language model. *arXiv preprint arXiv:2504.00698*, 2025.

[72] Jason Tsay, Laura Dabbish, and James Herbsleb. Influence of social and technical factors for evaluating contribution in GitHub. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, pages 356–366, 2014.

[73] Jens Tuyls, Dylan J. Foster, Akshay Krishnamurthy, and Jordan T. Ash. Representation-Based exploration for language models: From Test-Time to Post-Training. *arXiv preprint arXiv:2510.11686*, 2025.

[74] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformers reinforcement learning. `https://github.com/huggingface/trl`, 2020.

[75] Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen, Jianxin Yang, Zhenru Zhang, et al. Beyond the 80/20 rule: High-Entropy minority tokens drive effective reinforcement learning for LLM reasoning. *arXiv preprint arXiv:2506.01939*, 2025.

[76] Weixun Wang, Shaopan Xiong, Gengru Chen, Wei Gao, Sheng Guo, Yancheng He, Ju Huang, Jiaheng Liu, Zhendong Li, Xiaoyang Li, et al. Reinforcement learning optimization for Large-Scale learning: An efficient and User-Friendly scaling library. *arXiv preprint arXiv:2506.06122*, 2025.

[77] Zhenhailong Wang, Xuehang Guo, Sofia Stoica, Haiyang Xu, Hongru Wang, Hyeonjeong Ha, Xiusi Chen, Yangyi Chen, Ming Yan, Fei Huang, et al. Perception-Aware policy optimization for multimodal reasoning. *arXiv preprint arXiv:2507.06448*, 2025.

[78] Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, et al. RAGEN: Understanding Self-Evolution in LLM agents via Multi-Turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025.

[79] Shirley Wu, Michel Galley, Baolin Peng, Hao Cheng, Gavin Li, Yao Dou, Weixin Cai, James Zou, Jure Leskovec, and Jianfeng Gao. CollabLLM: From passive responders to active collaborators. *arXiv preprint arXiv:2502.00640*, 2025.

[80] Zhongwen Xu and Zihan Ding. Single-stream policy optimization. *arXiv preprint arXiv:2509.13232*, 2025.

[81] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[82] Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, and Jianfeng Gao. Your efficient RL framework secretly brings you off-policy RL training. `https://fengyao.notion.site/off-policy-rl`, August 2025.

[83] Deheng Ye, Zhao Liu, Mingfei Sun, Bei Shi, Peilin Zhao, Hao Wu, Hongsheng Yu, Shaojie Yang, Xipeng Wu, Qingwei Guo, et al. Mastering complex control in MOBA games with deep reinforcement learning. *arXiv preprint arXiv:1912.09729*, 2019.

[84] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. DAPO: An Open-Source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

[85] Yu Yue, Yufeng Yuan, Qiying Yu, Xiaochen Zuo, Ruofei Zhu, Wenyuan Xu, Jiaze Chen, Chengyi Wang, TianTian Fan, Zhengyin Du, et al. VAPO: Efficient and reliable reinforcement learning for advanced reasoning tasks. *arXiv preprint arXiv:2504.05118*, 2025.

[86] Kaichen Zhang, Yuzhong Hong, Junwei Bao, Hongfei Jiang, Yang Song, Dingqian Hong, and Hui Xiong. GVPO: Group variance policy optimization for large language model Post-Training. *arXiv preprint arXiv:2504.19599*, 2025.

[87] Yuzhong Zhao, Yue Liu, Junpeng Liu, Jingye Chen, Xun Wu, Yaru Hao, Tengchao Lv, Shaohan Huang, Lei Cui, Qixiang Ye, et al. Geometric-Mean policy optimization. *arXiv preprint arXiv:2507.20673*, 2025.

[88] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

[89] Haizhong Zheng, Jiawei Zhao, and Beidi Chen. Prosperity before collapse: How far can Off-Policy RL reach with stale data on LLMs? *arXiv preprint arXiv:2510.01161*, 2025.

[90] Siqi Zhou, Lukas Brunke, Allen Tao, Adam W. Hall, Federico Pizarro Bejarano, Jacopo Panerati, and Angela P. Schoellig. What is the impact of releasing code with publications? statistics from the machine learning, robotics, and control communities. *arXiv preprint arXiv:2308.10008*, 2023.

[91] Xuekai Zhu, Daixuan Cheng, Dinghuai Zhang, Hengli Li, Kaiyan Zhang, Che Jiang, Youbang Sun, Ermo Hua, Yuxin Zuo, Xingtai Lv, et al. FlowRL: Matching reward distributions for LLM reasoning. *arXiv preprint arXiv:2509.15207*, 2025.

[92] Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. slime: An LLM post-training framework for RL scaling. `https://github.com/THUDM/slime`, 2025.

A Labelling rules

Each paper receives exactly one label from its title and abstract.

An RL method is *general* when the component is the paper's main contribution and is presented for RL with language models at large. It is *embedded* when it serves a paper about one task, domain or application. A component that works only with a domain-specific reward makes the paper an application. Two included sources served as examples in an early version of these rules. Counting them, method recognition is 30 of 32.

Label	Meaning
RL method	A new or modified training-time RL component: loss or objective, advantage or baseline, clipping or trust region, importance weighting or off-policy correction, rollout sampling or filtering that feeds the gradient, task-agnostic reward shaping, KL or entropy regularisation, credit assignment, or an optimiser built for RL on language models.
RL application	Existing RL algorithms applied to a task or domain, including domain-specific rewards or verifiers.
Analysis	Empirical study, reproduction or ablation without a new method.
Systems	Training systems, asynchronous pipelines and throughput work with the objective unchanged.
Offline preference	Offline preference optimisation in the DPO family.
Distillation	The main objective matches a teacher.
Other	Surveys, benchmarks, inference-time methods, SFT objectives, generic reward-model fitting, and papers outside RL training of language models.

B Manual review

The review sample of 200 papers was drawn from the first labelled batches, stratified by label. Table 6 compares review and labels.

Table 6: Manual review against the labels.

Review \ Label	General	Embedded	Other
General	68	3	1
Embedded	6	12	32
Other	2	1	75

Within each label stratum, review found general methods in 68 of 76 papers labelled general, 3 of 16 labelled embedded, and 1 of 108 labelled otherwise. Weighting these rates by the size of each stratum in the pool gives about 1,590 general methods. The main disagreement lies between embedded methods and applications, a boundary outside the denominator.

C Full-text coding

The comparison covers every included general method from 2025 and a random sample of general methods from 2025 that no framework included. Controls were drawn within each quarter, in proportion to the quarterly counts of included papers at sampling time. The quarterly counts are 7, 10, 7 and 7 for included papers and 32, 54, 27 and 37 for controls. Papers were coded in a shuffled order without access to inclusion status, and every field was read from the first arXiv version. The included papers found by the citation audit were coded later from their first versions. A second reader recoded all of them blind, together with 16 controls, and the comparison uses the blind codes. On the recoded controls the two readers agree on size thresholds, mixture of experts, extra models, compute and affiliation, and differ most on code release and failure motivation. With the recoded controls substituted, the two main gaps compare against 14 of 141 and 12 of 149 controls. arXiv withdrew the first version of 1 control paper, and the comparison leaves it out. One coded change form was a placeholder for a prompt-only change and is treated as missing. Each paper was coded on the following fields.

- Affiliations on the first page, and whether the version links training code for the method.
- Whether the method trained a policy of at least 14B and of at least 30B parameters, and whether it trained a mixture of experts.
- The form of the change. A *coefficient* change alters a coefficient, clip, mask or weight in the existing loss. A *term* adds a loss term, estimator or sampling rule. A *pipeline* change adds a training stage, a model or a system component.
- The number of new hyperparameters, and whether the method needs an extra model.

- Whether the paper is motivated by an observed training failure.
- The evaluation domains, and whether compute is reported.

Fields a paper does not state were left empty and are excluded from the corresponding row of Figure 3. Papers without an empirical evaluation are excluded from the row on evaluation breadth.

Table 7: Sensitivity of the two largest gaps. Unit: paper. Each cell gives included papers against non-included papers. The last two rows leave out included papers that the framework of their own company includes, and then every paper from a company that maintains one of the frameworks. p is from a two-sided Fisher exact test on the two counts of the cell.

Analysis	Policy of 30B or more	p	Coefficient change	p
Main	12/29 vs 14/141	$<$0.001	9/31 vs 13/149	0.004
Including the exceptions	17/35 vs 14/141	$<$0.001	9/38 vs 13/149	0.020
Without model reports	12/29 vs 13/139	$<$0.001	9/31 vs 13/147	0.005
With an industry author	10/25 vs 11/90	0.003	7/26 vs 9/93	0.045
Missing values against gap	12/31 vs 23/150	0.005	9/31 vs 14/150	0.006
Threshold of 14B	13/29 vs 19/141	$<$0.001	–	
Before the citation audit	11/22 vs 14/141	$<$0.001	8/23 vs 13/149	0.002
Without own-company inclusion	9/24 vs 14/141	0.001	8/26 vs 13/149	0.004
Without framework companies	6/19 vs 9/119	0.007	6/21 vs 13/128	0.031

D Included papers outside the general rule

These 8 included papers do not meet the general rule from their abstracts, and the main rate leaves them out. Most are model reports or system papers that describe an included component only in the body. Keeping them out lets the denominator and the comparison of Section 6 use one eligibility rule.

Table 8: Included papers that do not meet the general rule from their abstracts. The attribution of masked importance sampling stays open among near-simultaneous sources.

Source	Abstract label	Model report	Included methods
Prime Intellect Team et al. [54]	RL method (embedded)	yes	Two-sided ratio clipping
He et al. [26]	RL application	yes	Adaptive entropy control
Fu et al. [20]	RL method (embedded)	no	Decoupled PPO objective
MiniMax et al. [49]	RL method (embedded)	yes	CISPO
Liu et al. [42]	RL method (embedded)	yes	AEPO
Liu et al. [44]	Analysis	no	LitePPO
Ling Team et al. [37]	RL method (embedded)	yes	Masked importance sampling (MIS)
DeepSeek-AI et al. [14]	RL application	yes	Off-policy sequence masking, Importance-weighted unbiased KL gradient

E The public record

The record covers 26,960 issues, pull requests and discussions downloaded through the GitHub API on 18 October 2026 (UTC), after the framework snapshots in Table 10. Every discussion was read in full, and none had more comments than one request returns. Formal review texts were read for the coded threads and the landing pull requests only. A thread matches a paper through a link to its arXiv, Hugging Face papers or alphaXiv page, its identifier after "arXiv:", its bare identifier, or its full title of at least five words. Identifiers are checked against the pool, so every match names a pool paper. The separate search by method name uses names of at least four characters with a capital letter that occur once in the pool, and every match for a paper without a record was read in its thread.

The citation audit reads every line added on each main branch and keeps the first commit that cites a paper in each file. All files citing a paper were checked by hand. Of the 23 general methods cited in

framework files but missed by parsing, 9 sit next to an implementation, including the NeMo-RL fix to the KL gradient and ROLL's trajectory-level agent pipeline. Another 10 appear in lists of projects or news, 2 in reference lists of technical report PDFs read as text, and 2 as a configuration note or a background reference.

Each proposal thread received one outcome and the reasons a maintainer stated explicitly in it, coded from the thread's own text. A first reader coded all threads, and a second reader coded them again blind to the first coding. The analysis uses the blind codes. The two codings agree on the outcome of 40 of 48 threads as raw agreement, and most differences concern redirects and withdrawals. Outcomes follow a fixed order. A proposer who closes the thread has withdrawn it. A maintainer who sends the proposal to an example, an experimental folder or a separate repository has redirected it. An explicit refusal is a decline, and a closure by a scheduled rule is a stale cleanup. When several threads in one framework concern the same paper, such as a withdrawn first attempt and its replacement, we code the main thread. Demand covers the number of requests, reactions and implementations elsewhere. Maintenance burden covers new options, trainers or interfaces to keep. Placement covers a move to an experimental folder, a recipe, an example or a separate repository. Evidence covers results to reproduce or gains to show. The other reasons are missing tests or documentation, an objection to the interface or architecture of the change, and an existing feature that already covers the case. A stated reason is also a concern in the review fields of the same thread, and one adjudication resolved the two threads where the codings first disagreed. The review fields of Table 3 were coded for every merged landing pull request and every proposal with one set of rules. A concern counts when someone other than the proposer raises it explicitly, and requests about style or naming do not count. An open concern is one the thread leaves unaddressed at its end, including follow-up work named at the merge.

The failure reading of Section 8 screened every issue and discussion opened since 1 January 2025 with keyword patterns for the thirteen failures of the catalog. The patterns were applied to the title and the first 800 characters of the opening post after removing code blocks, logs and configuration lines. One reader coded each of the 648 matches as a failure report with the failure named, a bug, a usage question, a feature request or other. A method request is a thread that asks for a named method. The recall sample draws 250 unscreened threads at random with a fixed seed, and the release keeps each coded thread's number, title, keywords and code. The report reading of Section 9 takes every model report in the pool for an open-weight model with 100B or more total parameters, reads its RL sections in the latest arXiv version, and keeps one quote per report with the methods it states. Reports that mention RL only in the body are outside the pool. Among the open-weight models with 100B or more parameters released in the window, the reports of Qwen3 [81], Pangu Ultra MoE [68], Intern-S1 [3], Command A [71], dots.llm1 [30] and Step-3 [67] fall outside it. This list comes from our own check of the models we know and not from a systematic search.

Table 9: Proposals to include general methods that no framework included. Unit: issue, pull request or discussion that proposes or requests the method, coded blind from its own text. The arXiv identifier tells apart methods with the same name. Reasons are those a maintainer stated in the thread.

Method	arXiv	Framework	Thread	Outcome	Stated reasons
VC-PPO decoupled GAE	2503.01491	verl	issue #479	open, under discussion	–
AGPO	2503.15952	TRL	PR #6368	redirected	maintenance burden, placement, evidence, already covered
JEPO	2503.19618	TRL	PR #5411	declined	maintenance burden, demand
CPPO (completion pruning)	2503.22342	TRL	issue #3291	open, no prior-merger reply	–
AdaRFT	2504.05520	verl	PR #1021	open, under discussion	–
SSR	2504.08837	TRL	PR #3496	closed as inactive	–
GRPO-LEAD	2504.09696	TRL	PR #6222	withdrawn by proposer	–
Turn-level advantage	2505.11821	verl	issue #1654	closed in stale cleanup	–
DisCO	2505.12366	verl	issue #3157	closed in stale cleanup	–

Method	arXiv	Framework	Thread	Outcome	Stated reasons
DisCO	2505.12366	verl	PR #3357	open, under discussion	–
KTAE	2505.16826	verl	issue #1783	closed in stale cleanup	–
Intuitor	2505.19590	TRL	PR #3507	withdrawn by proposer	–
VeriFree	2505.21493	TRL	issue #3777	declined	demand, maintenance burden
T-PPO	2506.15050	verl	PR #3728	open, no prior-merger reply	–
RLPR	2506.18254	TRL	issue #3928	declined	demand, placement, maintenance burden
ESSA	2507.04453	verl	PR #3637	open, no prior-merger reply	–
FR3E	2507.07017	OpenRLHF	issue #1093	open, no prior-merger reply	–
FR3E	2507.07017	verl	issue #2755	closed in stale cleanup	–
RLCR	2507.16806	TRL	issue #3871	declined	demand, maintenance burden, already covered
LightningRL	2508.03680	verl	issue #3434	closed in stale cleanup	–
GTPO	2508.03772	TRL	issue #5226	redirected	demand, maintenance burden, placement, design, already covered
Pass@k Training	2508.10751	verl	issue #3393	closed in stale cleanup	–
CHORD	2508.11408	slime	issue #1843	open, no prior-merger reply	–
CHORD	2508.11408	verl	issue #3223	closed in stale cleanup	–
HICRA	2509.03646	TRL	PR #4997	declined	demand, maintenance burden, design
PSPO	2509.21282	TRL	PR #4548	declined	demand, maintenance burden, tests and docs
Evolution Strategies	2509.24372	OpenRLHF	PR #1214	open, no prior-merger reply	–
GDPO	2510.08554	NeMo-RL	PR #3550	open, no prior-merger reply	–
SRL	2510.25992	TRL	issue #4759	declined	demand, maintenance burden, already covered
RLVE	2511.07317	slime	PR #1020	redirected	placement
HA-DW	2601.08521	TRL	PR #4872	closed as inactive	maintenance burden, placement, evidence, tests and docs
JustGRPO	2601.15165	NeMo-RL	PR #4258	open, no prior-merger reply	–
CFPO	2601.22801	OpenRLHF	PR #1184	open, no prior-merger reply	–
CFPO	2601.22801	TRL	PR #5027	declined	tests and docs
CFPO	2601.22801	verl	PR #5252	open, no prior-merger reply	–
F-GRPO	2602.06717	slime	issue #1565	redirected	maintenance burden, placement, evidence
F-GRPO	2602.06717	verl	issue #5256	withdrawn by proposer	–
F-GRPO	2602.06717	verl	PR #5334	open, no prior-merger reply	–

Method	arXiv	Framework	Thread	Outcome	Stated reasons
Dr. MAS	2602.08847	AReaL	issue #1114	closed in stale cleanup	–
HACPO	2603.02604	verl recipes	issue #143	open, no prior-merger reply	–
FIPO	2603.19835	slime	PR #1801	open, no prior-merger reply	–
TPO	2604.06159	TRL	PR #5591	closed without comment	–
SNR-Aware Filtering	2604.06268	TRL	PR #7242	open, no prior-merger reply	–
SNR-Aware Filtering	2604.06268	verl	PR #7894	open, under discussion	–
POISE	2605.07579	verl recipes	PR #157	open, no prior-merger reply	–
CoDaPO	2606.07950	verl	PR #7497	open, no prior-merger reply	–
CPPO	2606.10968	verl	PR #6731	open, under discussion	–
STARE	2606.19236	TRL	PR #6167	open, no prior-merger reply	–

F Framework selection

Table 10 applies the selection criteria of Section 3 to the open RL frameworks with at least 2,000 GitHub stars and gives the commit analysed in each. The recipe repository of verl is analysed at commit `81d782765e` of 2026-10-13.

Table 10: Open RL frameworks against the selection criteria of Section 3. Unit: repository with at least 2,000 stars on 19 October 2026. Commits are the snapshots of the seven tracked frameworks and the latest commits of the other qualifying ones. For qualifying frameworks outside the study, the count gives general methods that this commit implements and none of the seven include.

Repository	Stars	Commit	Status
`verl-project/verl`	23,926	`18cfb74852` 2026-10-16	tracked
`huggingface/trl`	19,551	`6d6901c5e3` 2026-10-16	tracked
`OpenRLHF/OpenRLHF`	10,097	`dc2a7ad326` 2026-09-17	tracked
`THUDM/slime`	8,686	`859938dcb2` 2026-10-17	tracked
`areal-project/AReaL`	5,843	`bb13e3bdb9` 2026-10-16	tracked
`alibaba/ROLL`	3,430	`d96d0cce56` 2026-10-13	tracked
`NVIDIA-NeMo/RL`	2,074	`83385307ab` 2026-10-17	tracked
`RLinf/RLinf`	5,439	`a78b6d7f64` 2026-10-17	meets the criteria; adds no general method
`allenai/open-instruct`	3,901	`ff32794a34` 2026-10-17	meets the criteria; adds 1 general method
`NovaSky-AI/SkyRL`	2,434	`2c54cefc7c` 2026-10-16	meets the criteria; adds no general method
`PrimeIntellect-ai/prime-rl`	2,175	`8dede158a9` 2026-10-19	meets the criteria; adds 1 general method
`unslothai/unsloth`	78,250		general fine-tuning; its RL trainers build on TRL
`hiyouga/LlamaFactory`	75,583		general fine-tuning
`huggingface/open-r1`	26,475		a recipe that trains with TRL
`modelscope/ms-swift`	15,886		general fine-tuning; its RL trainers build on TRL
`OpenPipe/ART`	10,813		trains through Unsloth and TRL
`rllm-org/rllm`	5,887		trains through verl
`meta-pytorch/torchtune`	5,818		general fine-tuning
`PrimeIntellect-ai/verifiers`	4,717		an environment library
`radixark/miles`	3,165		a fork of slime

G Supplementary tables and figures

Table 11: Option names at the snapshot for recent methods included in three or more frameworks. Unit: catalog method. Names are the registry keys, loss types, configuration fields or recipe folders that the parser maps to each method.

Method	Option names at the snapshot
Clip-higher	verl: `clip_ratio_high`, `recipe/dapo`; TRL: `epsilon_high`; OpenRLHF: `eps_clip_low_high`; NeMo-RL: `ratio_clip_max`; ROLL: `pg_clip_high`; slime: `eps_clip_high`; AReaL: `dapo_dynamic_bs`, `eps_clip_higher`
GSPO	verl: `gspo`; TRL: `importance_sampling_level`; OpenRLHF: `gspo`; NeMo-RL: `sequence_level_importance_ratios`; ROLL: `importance_sampling_seq`; slime: `gspo`; AReaL: `gspo`, `importance_sampling_level`
Dynamic sampling	verl: `filter_groups`, `recipe/dapo`; OpenRLHF: `dynamic_filtering`; NeMo-RL: `use_dynamic_sampling`; ROLL: `difficulty_mask`; slime: `dynamic_sampling_filter`; AReaL: `dapo_dynamic_bs`, `dynamic_bs`
Dr. GRPO	verl: `norm_adv_by_std_in_grpo`, `seq-mean-token-sum-norm`; TRL: `dr_grpo`, `scale_rewards`; OpenRLHF: `dr_grpo`, `no_std_norm`; ROLL: `seq-mean-token-sum-norm`; slime: `disable_grpo_std_normalization`; AReaL: `drgrpo`
CISPO	verl: `cispo`; TRL: `cispo`; NeMo-RL: `use_cispo`; ROLL: `cispo`; slime: `cispo`; AReaL: `use_cispo_loss`
Truncated importance sampling (TIS)	verl: `rollout_is`; TRL: `vllm_importance_sampling_correction`; OpenRLHF: `is_correction`; NeMo-RL: `truncated_importance_sampling_ratio`; ROLL: `tis`, `train_infer_correction`; slime: `use_tis`
Masked importance sampling (MIS)	verl: `rollout_rs`; TRL: `vllm_importance_sampling_mode`; OpenRLHF: `is_correction_mask`; NeMo-RL: `seq_logprob_error_threshold`, `seq_mask_tis`; ROLL: `train_infer_filter`; AReaL: `icepop`, `rejection_sampling`
REINFORCE++	verl: `reinforce_plus_plus`, `reinforce_plus_plus_baseline`; OpenRLHF: `reinforce`, `reinforce_baseline`; NeMo-RL: `reinforce_plus_plus`; slime: `reinforce_plus_plus`, `reinforce_plus_plus_baseline`; AReaL: `reinforce_baseline`
Token-level loss aggregation	verl: `recipe/dapo`; TRL: `dapo`; NeMo-RL: `token_level_loss`; ROLL: `token-mean`; slime: `calculate_per_token_loss`
Overlong filtering and penalty	verl: `overlong_buffer`, `recipe/dapo`; TRL: `mask_truncated_completions`; OpenRLHF: `overlong_buffer`; NeMo-RL: `overlong_filtering`; AReaL: `overlong_reward_penalty`
Rollout routing replay	verl: `enable_rollout_routing_replay`; NeMo-RL: `router_replay`; ROLL: `enable_rollout_routing_replay`; slime: `use_rollout_routing_replay`; AReaL: `enable_router_replay`
SAPO	verl: `sapo`; TRL: `sapo`; AReaL: `sapo`, `use_sapo_loss`
GDPO	verl: `gdpo`; TRL: `multi_objective_aggregation`; NeMo-RL: `gdpo`

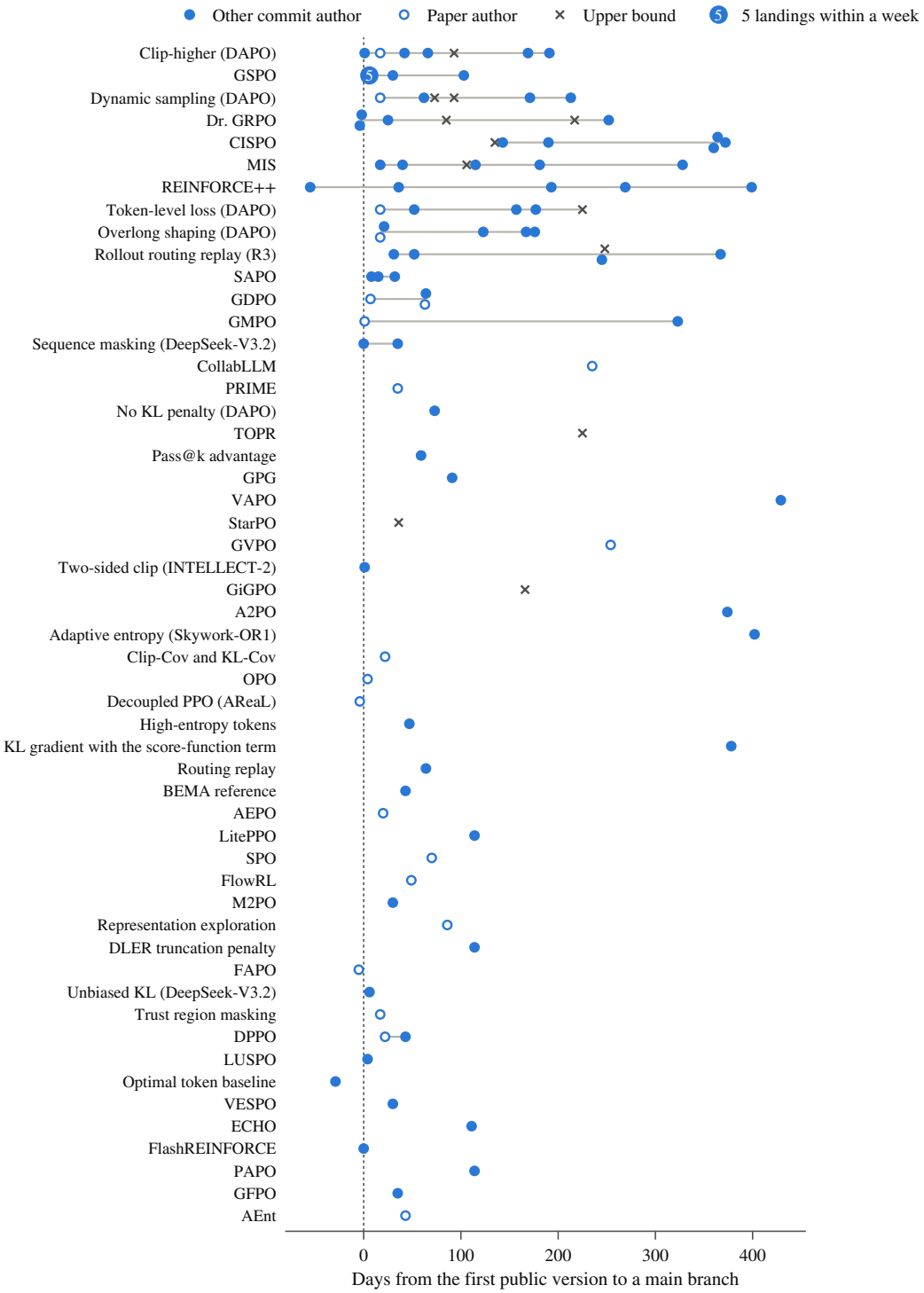


Figure 5: Time from first public version to a main branch, all methods. Unit: catalog method with a dated source from 2025 or later. Marks as in Figure 2.

H Method catalog

Table 12: Method catalog. Unit: catalog method. First landing is the earliest main-branch commit in any framework, with $\leq$ for upper bounds. Frameworks list where the method is offered at the snapshot.

Method	Source	First public	First landing	Frameworks now
PPO (clipped surrogate, GAE)	Schulman et al. [59]	2017-07-20	2022-12-30	verl, OpenRLHF, NeMo-RL, ROLL, slime, AReaL
Dual-clip PPO	Ye et al. [83]	2019-12-20	2025-04-02	verl, OpenRLHF, NeMo-RL, ROLL, slime, AReaL
k3 (low-variance) KL estimator	Schulman [58]	2020-03-07	2025-01-23	verl
ReMax	Li et al. [36]	2023-10-16	2025-02-10	verl
GRPO	Shao et al. [61]	2024-02-05	2025-01-20	verl, TRL, OpenRLHF, NeMo-RL, ROLL, slime, AReaL
RLOO	Ahmadian et al. [1]	2024-02-22	2024-05-22	verl, TRL, OpenRLHF, NeMo-RL, AReaL
Direct Reward Optimisation (quadratic log-ratio penalty)	Richemond et al. [56]	2024-05-29	2026-08-04	verl
REINFORCE++	Hu et al. [28]	2025-01-04	2024-11-10	verl, OpenRLHF, NeMo-RL, slime, AReaL
CollabLLM (multiturn-aware rewards)	Wu et al. [79]	2025-02-02	2025-09-25	verl
PRIME (implicit process rewards)	Cui et al. [10]	2025-02-03	2025-03-10	verl
Clip-higher (decoupled clip range)	Yu et al. [84]	2025-03-18	2025-03-19	verl, TRL, OpenRLHF, NeMo-RL, ROLL, slime, AReaL
Dynamic sampling (filter zero-variance groups)	Yu et al. [84]	2025-03-18	2025-04-04	verl, OpenRLHF, NeMo-RL, ROLL, slime, AReaL
Remove the KL penalty for reasoning RL	Yu et al. [84]	2025-03-18	2025-05-30	TRL
Overlong filtering and penalty	Yu et al. [84]	2025-03-18	2025-04-04	verl, TRL, OpenRLHF, NeMo-RL, AReaL
Token-level loss aggregation	Yu et al. [84]	2025-03-18	2025-04-04	verl, TRL, NeMo-RL, ROLL, slime
TOPR (tapered off-policy REINFORCE)	Roux et al. [57]	2025-03-18	$\leq$2025-10-29	ROLL
Pass@k advantage	Tang et al. [70]	2025-03-25	2025-05-23	verl
Dr. GRPO (no length or std normalisation)	Liu et al. [43]	2025-03-26	2025-03-22	verl, TRL, OpenRLHF, ROLL, slime, AReaL
GPG	Chu et al. [9]	2025-04-03	2025-07-03	verl
VAPO (decoupled and length-adaptive GAE, positive-example NLL)	Yue et al. [85]	2025-04-07	2026-06-10	NeMo-RL
StarPO (trajectory-level optimisation for multi-turn agents, RAGEN)	Wang et al. [78]	2025-04-24	$\leq$2025-05-30	ROLL
GVPO (group variance policy optimisation)	Zhang et al. [86]	2025-04-28	2026-01-07	verl
Two-sided ratio clipping (INTELLECT-2)	Prime Intellect Team et al. [54]	2025-05-12	2025-05-13	TRL
GiGPO (group-in-group credit for agents)	Feng et al. [18]	2025-05-16	$\leq$2025-10-29	ROLL
A2PO (optimal advantage regression)	Brantley et al. [7]	2025-05-27	2026-06-05	TRL
Clip-Cov and KL-Cov	Cui et al. [11]	2025-05-28	2025-06-19	verl

Method	Source	First public	First landing	Frameworks now
Adaptive entropy control (Skywork-OR1)	He et al. [26]	2025-05-28	2026-07-04	TRL
OPO (optimal reward baseline)	Hao et al. [25]	2025-05-29	2025-06-02	verl
Decoupled PPO objective (AReaL)	Fu et al. [20]	2025-05-30	2025-05-26	AReaL
Train on high-entropy tokens only	Wang et al. [75]	2025-06-02	2025-07-19	TRL
KL gradient with the score-function term (Tang and Munos)	Tang and Munos [69]	2025-06-11	2026-06-24	NeMo-RL
CISPO	MiniMax et al. [49]	2025-06-16	$\leq$2025-10-29	verl, TRL, NeMo-RL, ROLL, slime, AReaL
PAPO (implicit perception loss)	Wang et al. [77]	2025-07-08	2025-10-30	removed
GSPO (sequence-level ratio)	Zheng et al. [88]	2025-07-24	2025-07-27	verl, TRL, OpenRLHF, NeMo-RL, ROLL, slime, AReaL
Routing replay across updates	Zheng et al. [88]	2025-07-24	2025-09-26	slime
GMPO (geometric-mean objective)	Zhao et al. [87]	2025-07-28	2025-07-29	verl, TRL
Bias-corrected EMA reference model	Block and Zhang [6]	2025-07-31	2025-09-12	TRL
Truncated importance sampling (TIS)	Yao et al. [82]	2025-08	2025-08-06	verl, TRL, OpenRLHF, NeMo-RL, ROLL, slime
AEPO (adaptive exploration policy optimisation)	Liu et al. [42]	2025-08-07	2025-08-27	verl
LitePPO (group-mean, batch-std normalisation with token-level loss)	Liu et al. [44]	2025-08-11	2025-12-03	AReaL
GFPO (group filtered policy optimisation)	Shrivastava et al. [65]	2025-08-13	2025-09-17	removed
AEnt (clamped entropy bonus with adaptive coefficient)	Shen [63]	2025-09-03	2025-10-16	removed
SPO (single-stream policy optimisation)	Xu and Ding [80]	2025-09-16	2025-11-25	verl
FlowRL (reward distribution matching)	Zhu et al. [91]	2025-09-18	2025-11-06	verl
M2PO (second-moment trust region for stale data)	Zheng et al. [89]	2025-10-01	2025-10-31	AReaL
Rollout routing replay (R3)	Ma et al. [45]	2025-10-13	2025-11-13	verl, NeMo-RL, ROLL, slime, AReaL
Representation-based exploration	Tuyls et al. [73]	2025-10-13	2026-01-07	verl
DLER truncation penalty	Liu et al. [40]	2025-10-16	2026-02-07	NeMo-RL
Masked importance sampling (MIS)	Ling Team et al. [37]	2025-10-21	2025-11-07	verl, TRL, OpenRLHF, NeMo-RL, ROLL, AReaL
FAPO	Ding et al. [16]	2025-10-26	2025-10-21	verl
SAPO (soft adaptive gating)	Gao et al. [21]	2025-11-25	2025-12-03	verl, TRL, AReaL
Off-policy sequence masking (DeepSeek-V3.2)	DeepSeek-AI et al. [14]	2025-12-02	2025-12-02	TRL, slime
Importance-weighted unbiased KL gradient (DeepSeek-V3.2)	DeepSeek-AI et al. [14]	2025-12-02	2025-12-08	TRL
Trust region masking (sequence max divergence)	Li et al. [34]	2025-12-28	2026-01-14	verl
GDPO (decoupled per-reward normalisation)	Liu et al. [41]	2026-01-08	2026-01-15	verl, TRL, NeMo-RL

Method	Source	First public	First landing	Frameworks now
DPPO (divergence-based trust region)	Qi et al. [55]	2026-02-04	2026-02-26	verl
LUSPO (length-unbiased sequence policy optimisation)	Liu et al. [39]	2026-02-05	2026-02-09	TRL
Optimal token baseline with variance proxy	Li et al. [35]	2026-02-06	2026-01-08	verl
VESPO	Shen et al. [62]	2026-02-11	2026-03-13	TRL
ECHO (auxiliary world-model loss on observation tokens)	Shrivastava et al. [66]	2026-05-23	2026-09-11	verl
FlashREINFORCE (single-rollout async RL)	framework	2026-09-14	2026-09-14	OpenRLHF
GRPO with replay buffer	framework	–	2025-09-24	removed
KPop (binary-KL token masking for rollout-training mismatch)	framework	–	2026-06-15	AReaL
Untruncated importance weights between rollout and trainer log-probs	framework	–	2025-04-22	NeMo-RL
SAO (direct double-sided importance sampling, ROLL)	framework	–	$\leq$2026-09-29	ROLL